

Analytical Appendix

Diagnosing Growth Constraints at The Look

Documented by Vo Thanh Nhu

MAIN PROBLEM

Revenue growth is slowing, and the company lacks clarity on the underlying drivers and **actionable levers for sustainable growth**.

DATA PREPARATION & MODELING

I. Objective

To ensure analytical consistency and efficiency at scale, a dedicated analytical dataset was constructed by consolidating transactional and dimensional data into a unified structure, followed by data cleaning and business-oriented feature engineering.

II. Data Validation

To ensure data integrity and analytical reliability, at both pre- and post-transformation step, targeted validation was applied to key fields that were most likely impacted by the step.

1. Pre-transformation validation (raw data profiling)

Initial validation was conducted on source tables to assess data completeness and structural integrity. Key checks included:

- Missing values in critical fields (order_id, user_id, product_id, pricing fields)
- Uniqueness of primary identifiers (user_id, product_id, order_id)
- Basic data consistency checks (e.g., presence of required timestamps)

Key SQL Excerpt – Missing values

```
SELECT
```

```
COUNT(*) AS total_row,  
  
COUNTIF(order_id IS NULL) AS null_order,  
  
COUNTIF(user_id IS NULL) AS null_user,  
  
COUNTIF(product_id IS NULL) AS null_product,  
  
COUNTIF(inventory_item_id IS NULL) AS null_inventory,  
  
COUNTIF(status IS NULL) AS null_status,  
  
COUNTIF(sale_price IS NULL) AS null_sale_price
```

```
FROM `bigquery-public-data.thelook_ecommerce.order_items`;
```

Note

Critical transactional fields were validated for completeness to ensure downstream metrics such as revenue and retention are not impacted by missing data.

Key SQL Excerpt – Uniqueness of identifiers

```
SELECT 'user' AS table_name,  
  
COUNT(*) AS total_id,  
  
COUNT(DISTINCT id) AS unique_id,  
  
COUNT(*)-COUNT(DISTINCT id) AS duplicate_id  
  
FROM bigquery-public-data.thelook_ecommerce.users
```

```
UNION ALL
```

```
SELECT 'product',  
  
COUNT(*) AS total_id,  
  
COUNT(DISTINCT id) AS unique_id,  
  
COUNT (*)-COUNT(DISTINCT id) AS duplicate_id
```

```
FROM bigquery-public-data.thelook_ecommerce.products
```

```
UNION ALL
```

```
SELECT 'order',  
COUNT (*) AS total_id,  
COUNT(DISTINCT order_id) AS unique_id,  
COUNT(*)-COUNT(DISTINCT order_id) AS duplicate_id  
FROM bigquery-public-data.thelook_ecommerce.orders;
```

Note

Ensuring uniqueness of primary identifiers helps prevent duplication issues during joins and aggregation steps.

2. Post-transformation validation

After data consolidation, filtering, and feature engineering, targeted validation checks were applied to ensure transformations did not introduce inconsistencies. Key checks included:

- Row count consistency after joins and filters
- Detection of duplicate records after joins
- Detection of newly introduced null values

Key SQL Excerpt

```
SELECT  
COUNT(*) - COUNT(DISTINCT order_item_id) AS duplicate_rows,  
COUNT(CASE WHEN category IS NULL THEN 1 END) AS null_category,  
COUNT(CASE WHEN order_status = 'Cancelled' THEN 1 END) AS  
remaining_cancelled_orders,  
FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`;
```

III. Data consolidation & modeling

The analysis uses multiple tables as sources, including:

- order_items (transaction-level data)
- products (product attributes)
- inventory_items (inventory lifecycle data)
- users (customer attributes)

To enable consistent analysis, these tables were consolidated into a single dataset at the **order-item** level, using table order_items as the base.

This method ensures:

- Preservation of all transactional records
- Alignment of granularity across all metrics
- Efficient querying without repeated joins

IV. Data Integration Logic

A series of LEFT JOIN commands was applied to enrich transactional data with product, inventory, and customer attributes, while ensuring no loss of records from the base table.

Key SQL Excerpt

```
FROM bigquery-public-data.thelook_ecommerce.order_items AS oi
```

```
LEFT JOIN bigquery-public-data.thelook_ecommerce.products AS p ON  
oi.product_id = p.id
```

```
LEFT JOIN bigquery-public-data.thelook_ecommerce.inventory_items AS ii ON  
oi.inventory_item_id = ii.id
```

```
LEFT JOIN bigquery-public-data.thelook_ecommerce.users AS u ON oi.user_id  
= u.id;
```

Note

Preserving order-level transactions by using order_item as the base table,

while additional attributes were appended via LEFT JOIN commands to avoid accidental data loss.

V. Feature engineering

Several resultant features were created across pricing, inventory, customer, and behavior dimension to translate raw data into decision-driven business insights.

1. Pricing feature

- Discount percentage
- Discount tier segmentation

Key SQL Excerpt

```
ROUND(((retail_price-sale_price)/retail_price)*100, 2) AS
discount_percentage,

CASE

WHEN retail_price = sale_price THEN 'Full Price'

WHEN (retail_price - sale_price) <= 0.2 THEN 'Light Discount'

WHEN (retail_price -sale_price) <= 0.5 THEN 'Mid Discount'

ELSE 'High Discount'

END AS discount_tier,
```

Finding

Discount levels were found to be consistently zero across the dataset, indicating no pricing variation between retail and sale prices.

Insight

Pricing does not appear to be a key lever influencing customer behavior or revenue performance in this dataset.

ð Given the absence of pricing variation, subsequent analysis focuses on alternative growth drivers such as customer retention and product performance.

2. Customer segmentation feature

- Customer type (repeat vs one-time)

Key SQL Excerpt

```
CASE WHEN oc.unique_order_count = 1 THEN 'One time buyer' ELSE 'Repeat buyer' END AS customer_type,
```

Note

Customer segmentation is derived from transactional behavior (order frequency), enabling downstream retention and behavioral analysis.

VI. Data cleaning

To ensure data quality and analytical reliability, filtering rules were applied to exclude invalid or non-relevant records:

- Remove cancelled transactions
- Exclude zero or negative pricing
- Ensure availability of key timestamps

Key SQL Excerpt

```
WHERE status != 'Cancelled'  
  
AND sale_price >0  
  
AND retail_price >0  
  
AND inventory_entry_date IS NOT NULL  
  
AND created_at IS NOT NULL;
```

Note

Data filtering removes invalid or incomplete records that could distort key metrics such as revenue, return rates, and inventory turnover.

VII. Data pipeline design

To simulate real-world data workflows, a two-layer structure was implemented:

- Raw Layer (summary_table) consolidates data from multiple sources
- Clean Layer (enriched_and_filtered_data view) applies filtering and

feature engineering

Note

This layered approach separates raw data from processed data, improving maintainability and reflecting real-world data pipeline design where datasets are continuously updated.

VIII. Analytical readiness

The new dataset maintains consistent order-item granularity while adding pre-computed business features.

→ Optimized for downstream analysis (retention, revenue, inventory).

DATA ANALYSIS

I. Growth Dianosis

Q1. How has revenue performance evolved over time?

Approach

Analyze monthly revenue trends and evaluate the month-over-month (MoM) growth rate to identify changes in growth dynamics over time.

Metric

- Monthly revenue = $\text{SUM}(\text{sale_price})$
- MoM growth rate = $\frac{(\text{current month revenue} - \text{previous month revenue})}{\text{previous month revenue}}$

Key SQL Excerpt

```
WITH monthly_revenue AS (  
    SELECT order_month,  
    ROUND(SUM (sale_price),2) AS revenue  
    FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`  
    GROUP BY 1)
```

```

SELECT

FORMAT_TIMESTAMP ('%m-%Y', order_month) AS month,

revenue,

ROUND((revenue - LAG(revenue) OVER(ORDER BY
order_month))/NULLIF(LAG(revenue) OVER (ORDER BY order_month),0),2) AS
MoM_growth_rate

FROM monthly_revenue

WHERE order_month >=

(SELECT DATE_SUB(MAX(order_month), INTERVAL 36 MONTH)

FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`)

ORDER BY order_month DESC;

```

Note

Narrow down the scope of data to only 3 latest years to identify trends easily. Additionally, time filtering is based on the latest available data to ensure analytical relevance.

Finding

Revenue growth shows monthly volatility but a sustained upward trajectory over time.

Insight

Growth is not sustainable, likely driven by a mix of short-term spikes (e.g., promotions) and weakening underlying demand.

Q2. Is the revenue volatility driven by changes in customer dynamics?

Approach

Revenue is decomposed into three core components:

- Customer acquisition (new customers)
- Customer retention (repeat customers)

- Customer value (average order value)

This decomposition helps identify which driver contributes most to revenue volatility.

Metrics

- New customer rate = new customers / total customers
- Repeat customer rate = repeat customers / total customers
- AOV = total revenue / total orders

Key SQL Excerpt

WITH

```
base_table AS (  
  
SELECT order_month, user_id, order_id, sale_price  
  
FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`  
  
),
```

```
first_purchase AS (  
  
SELECT user_id,  
  
MIN(order_month) AS first_time_buying  
  
FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`  
  
GROUP BY 1),
```

```
customer_label AS (  
  
SELECT order_month, b.user_id, sale_price, order_id,  
  
CASE WHEN first_time_buying = order_month THEN 'New'  
  
ELSE 'Repeat'  
  
END AS customer_classification  
  
FROM base_table AS b
```

```

LEFT JOIN first_purchase AS fp ON b.user_id = fp.user_id),

monthly_metric AS (

SELECT

order_month,

COUNT(DISTINCT CASE WHEN customer_classification = 'New' THEN user_id END)
AS new_customer,

COUNT(DISTINCT CASE WHEN customer_classification = 'Repeat' THEN user_id
END) AS repeat_customer,

COUNT(DISTINCT user_id) AS total_customer,

COUNT(DISTINCT order_id) AS total_order,

SUM(sale_price) AS revenue

FROM customer_label

GROUP BY 1)

SELECT

order_month,

ROUND(((new_customer/total_customer)*100),2) AS customer_acquisition_rate,

ROUND (((repeat_customer/total_customer)*100),2) AS
customer_retention_rate,

ROUND((revenue/total_order),2) AS AOV,

revenue

FROM monthly_metric

ORDER BY 1 DESC;

```

Finding

Customer acquisition remains strong, but retention is consistently low.

Insight

Weak retention is likely the primary driver limiting sustainable growth,

despite continued new customer acquisition.

To validate this hypothesis, it is critical to examine customer quality – specifically, value distribution and behavioral patterns across one-time and repeat buyers.

II. Customer Retention

Q3. To what extent is customer value concentrated among repeat vs one-time buyers?

Approach

Customer quality is evaluated by analyzing the composition of the customer base and how value is distributed across segments.

Metrics

- % one-time buyers
- % repeat buyers
- Revenue contribution by segment

Key SQL Excerpt

WITH

customer_base AS (

SELECT

customer_type_lifetime,

COUNT (DISTINCT user_id) AS total_customer,

SUM(sale_price) AS revenue,

FROM `just-cosmos-479109-p7.thelook\_clean.enriched\_and\_filtered\_data`

GROUP BY 1)

SELECT

customer_type_lifetime,

ROUND (total_customer*1.0/SUM(total_customer) OVER (),2) AS

customer_percentage,

ROUND (revenue*1.0/SUM(revenue) OVER (),2) AS revenue_contribution,

FROM customer_base;

Finding

Repeat customers contribute a disproportionate share of revenue despite representing a smaller share of the customer base.

Insight

Customer value is highly concentrated among repeat buyers, highlighting retention as a critical growth lever.

However, lifetime segmentation alone does not explain the drivers behind this value gap, thus requiring a deeper analysis of purchasing behaviors across the customer lifecycle.

Q4. Is repeat customer value driven more by purchase frequency or by stronger purchasing behavior?

Approach

Purchasing behavior is analyzed across the customer lifecycle by distinguishing one-time buyers, first-time purchases, and repeat purchases. This enables identification of behavioral changes over time (first-time vs repeat)

The analysis focuses on whether the value advantage of repeat customers is driven by higher purchase frequency or by improvements in per-order behavior.

Metrics

Core drivers

Purchase frequency (orders per customer)

Behavior evolution (lifecycle insight)

- AOV: first-time vs repeat
- Basket size: first-time vs repeat
- Return rate: first-time vs repeat

Key SQL Excerpt

WITH

```
order_sequence AS (  
    SELECT order_id, user_id, sale_price, order_status, customer_type_lifetime,  
    ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY created_at) AS order_num,  
    FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`  
),  
order_relabeling AS (  
    SELECT *,  
    CASE WHEN customer_type_lifetime = 'One time buyer' THEN '1-time'  
    WHEN order_num = 1 THEN '1st-time'  
    ELSE 'Repeat'  
    END AS segment  
    FROM order_sequence  
),  
customer_level AS (  
    SELECT user_id, segment,  
    COUNT(DISTINCT order_id) AS total_order_pc,  
    SAFE_DIVIDE(COUNT (*), COUNT(DISTINCT order_id)) AS basket_size_pc,  
    MAX(CASE WHEN order_status = 'Returned' THEN 1 ELSE 0 END) AS  
returned_order,  
    SAFE_DIVIDE(SUM(sale_price), COUNT(DISTINCT order_id)) AS AOV_pc,  
    FROM order_relabeling  
    GROUP BY 1, 2)  
SELECT  
    segment,  
    ROUND(AVG(basket_size_pc), 2) AS avg_basket_size_pc,
```

```
ROUND(AVG(AOV_pc),2) AS avg_AOV_pc,  
ROUND(AVG(total_order_pc),2) AS avg_order_pc,  
ROUND(AVG(returned_order),2) AS avg_return_rate,  
FROM customer_level  
GROUP BY 1;
```

Note

Segmentation is fixed because lifetime segmentation may include first purchases into repeat customers, potentially smoothing behavioral differences.

Finding

Repeat customers generate higher value via frequency and basket expansion. Notably, one-time buyers exhibit stronger initial purchasing behavior than first-time buyers but do not return.

Insight

Repeat customer value is primarily driven by higher purchase frequency and basket expansion rather than stronger purchasing behavior, highlighting retention as the key constraint to growth.

Q5. Where in the post-purchase lifecycle do customers fail to convert into repeat buyers?

Approach

Cohort analysis is used to track customer retention after the first purchase, identifying where customers drop off across subsequent purchase stages and how engagement decays over time.

Metric

- Retention rate by cohort (purchase stage or time since first purchase)
- Drop-off rate after first purchase (1st → 2nd order conversion)
- Retention decay across subsequent purchases (2nd → 3rd, etc.)

Key SQL Excerpt

WITH

user_cohort AS (

SELECT user_id,

MIN (order_month) AS first_purchase_month

FROM `just-cosmos-479109-p7.thelook\_clean.enriched\_and\_filtered\_data`

GROUP BY 1

),

month_index AS (

SELECT uc.user_id, uc.first_purchase_month, b.order_month,

DATE_DIFF(b.order_month,uc.first_purchase_month,MONTH) AS month_num,

ROW_NUMBER() OVER (PARTITION BY uc.user_id ORDER BY created_at) AS
order_sequence,

FROM user_cohort AS uc

JOIN `just-cosmos-479109-p7.thelook\_clean.enriched\_and\_filtered\_data` AS b
ON uc.user_id = b.user_id

),

cohort_size AS (

SELECT first_purchase_month,

COUNT (DISTINCT user_id) AS cohort_size,

FROM month_index

GROUP BY 1),

retention_through_time AS (

SELECT first_purchase_month, month_num,

COUNT(DISTINCT user_id) AS active_customer,

COUNT(DISTINCT CASE WHEN order_sequence = 2 THEN user_id END) AS
customer_2nd_order

FROM month_index

GROUP BY 1,2

```

)

SELECT

    cs.first_purchase_month, r.month_num, cs.cohort_size, r.active_customer,

    ROUND(SAFE_DIVIDE(r.active_customer, cs.cohort_size)*100,2) AS
retention_rate,

    ROUND(SAFE_DIVIDE(customer_2nd_order, cs.cohort_size)*100,2) AS
conversion_rate,

    ROUND(SAFE_DIVIDE(r.active_customer, LAG(r.active_customer) OVER(PARTITION
BY cs.first_purchase_month ORDER BY r.month_num))*100,2) AS decay_rate

FROM cohort_size AS cs

JOIN retention_through_time AS r ON cs.first_purchase_month =
r.first_purchase_month

WHERE r.month_num <= 12

ORDER BY 1 DESC, 2 ASC;

```

Note

Recent cohorts are right-censored, as they have had less time to generate repeat purchases, which may understate their true retention potential.

Finding

Customer retention is highly front-loaded and volatile, with significant drop-off occurring within the first 1–3 months and retention falling to negligible levels (~1%) thereafter.

Insight

The business relies on acquisition to drive revenue, while structurally weak retention constrains long-term growth and customer value.

ð Headline: Customer value collapses after the first purchase, forcing growth to rely on continuous acquisition.

Q6. When do repeat customers repurchase, and what categories drive their repeat behavior?

Approach

Analyze repeat customer behavior across two dimensions:

- Time between purchases to identify repurchase timing patterns
- Product categories to understand where repeat demand is concentrated.

Metrics

Repurchase timing (core behavior)

- Average interpurchase time (average number of days between consecutive purchases)

Repeat activity

- Number of repeat orders
- Number of repeat customers

Category preference

- Top categories by repeat users
- Repeat orders by category
- Orders per user (repeat intensity)

Key SQL Excerpt

WITH

```
repurchase_base AS (  
    SELECT user_id, order_id, category, order_month,  
    CAST(created_at AS DATE) AS order_date,  
    ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY created_at) AS  
    order_sequence  
FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`  
WHERE customer_type_lifetime = 'Repeat buyer'),
```

```
interpurchase AS (  
    SELECT *,
```

```
DATE_DIFF(order_date, LAG(order_date) OVER(PARTITION BY user_id ORDER BY
order_date), DAY) AS day_diff,

FROM repurchase_base),
```

```
category_prefer AS (

SELECT category,

COUNT(DISTINCT user_id) AS repeat_user,

COUNT(DISTINCT order_id) AS repeat_order,

ROUND(SAFE_DIVIDE(COUNT(DISTINCT order_id), COUNT(DISTINCT user_id)), 2)
AS orders_per_user,

RANK() OVER(ORDER BY ROUND(SAFE_DIVIDE(COUNT(DISTINCT order_id),
COUNT(DISTINCT user_id)), 2) DESC) AS category_rank

FROM repurchase_base

GROUP BY category)
```

```
SELECT

(SELECT ROUND(AVG(day_diff), 2) FROM interpurchase WHERE day_diff IS NOT
NULL) AS avg_interpurchase,

(SELECT ARRAY_AGG(STRUCT (category, repeat_user, repeat_order,
orders_per_user)) FROM category_prefer WHERE category_rank <= 3) AS
top_3_category;
```

Note

The analysis considers both number of repeat customers and repeat orders to evaluate customer stickiness regarding category preference.

Finding

Repeat purchases occur infrequently (~193-day cycle) and lack depth (~1.1 orders/user), even in core categories.

Insight

Even Repeat behavior is neither frequent nor intensive, indicating that weak engagement fails to compound into meaningful long-term value.

→ The lack of strong repeat engagement suggests potential frictions in the customer experience, with returns and product mismatch emerging as key hypotheses to explain weak retention.

III. Revenue Leakage

Q7. How significant is revenue loss due to returns?

Approach

Quantify the magnitude of revenue loss from returns and assess its significance relative to total revenue.

Metric

- Returned revenue
- Total revenue
- Return rate (revenue-based and order_based)

Key SQL Excerpt

SELECT

```
SUM (CASE WHEN order_status = 'Returned' THEN sale_price END) AS  
returned_revenue,
```

```
SUM (sale_price) AS total_revenue,
```

```
ROUND(SAFE_DIVIDE(SUM (CASE WHEN order_status = 'Returned' THEN sale_price  
END), SUM (sale_price)), 2) AS return_rate
```

```
FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`;
```

Finding

Returns account for approximately **12% of total revenue**.

Insight

Returns constitute a material revenue leakage, suggesting that underlying product or experience issues may be suppressing customer retention.

ð This suggests that improving product fit and reducing return rates could unlock a meaningful portion of lost revenue.

Q8. Which products/categories drive the highest return rates?

Approach

Quantify the financial impact of returns by measuring returned revenue relative to total revenue and analyzing how this impact evolves over time.

Metric

Return rate (returned items / total sold items)

Key SQL Excerpt

```
SELECT category,  
  
    SUM (CASE WHEN order_status = 'Returned' THEN sale_price END) AS  
    returned_revenue,  
  
    ROUND(SAFE_DIVIDE(SUM (CASE WHEN order_status = 'Returned' THEN sale_price  
END), SUM (sale_price)), 2) AS return_rate,  
  
    ROUND(SAFE_DIVIDE(SUM (CASE WHEN order_status = 'Returned' THEN sale_price  
END), SUM(SUM(sale_price)) OVER ()), 2) AS contribution_to_total_sale  
  
FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`  
  
GROUP BY 1  
  
ORDER BY return_rate DESC;
```

Finding

Fit-sensitive categories show the highest return rates, while outerwear drives the largest revenue loss from returns.

Insight

Returns are driven by two distinct mechanisms: high return propensity in fit-sensitive categories and high revenue exposure in higher-value outerwear.

- Returns in body-hugging products could be attributed to sizing problems, highlighting sizing and product fit as key levers to reduce revenue leakage.
- Outerwear purchases may be more occasion-driven, increasing the risk of post-use returns or lower long-term satisfaction.

IV. Product & Inventory

Q9. Which product categories underperform in terms of demand and revenue contribution?

Approach

Evaluate category performance by comparing demand (units sold) and revenue contribution to identify categories with weak commercial traction.

Metric

Demand

- Units sold by category
- Sales share (%) by category (units-based)

Revenue

- Revenue by category
- Revenue share (%) by category

Average selling price (ASP)

Key SQL Excerpt

```
WITH each_category AS (  
    SELECT category,  
    COUNT (DISTINCT CASE WHEN order_status != 'Returned' THEN order_id END) AS  
sold_unit,  
    SUM(CASE WHEN order_status != 'Returned' THEN sale_price END) AS revenue  
FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`  
GROUP BY 1)  
  
SELECT  
category,  
sold_unit,
```

```

ROUND(SAFE_DIVIDE(sold_unit, SUM(sold_unit) OVER()), 2) AS sale_share,
revenue,
ROUND(SAFE_DIVIDE(revenue, SUM(revenue) OVER()), 2) AS revenue_share,
ROUND(SAFE_DIVIDE(revenue, sold_unit), 2) AS avg_selling_price
FROM each_category
ORDER BY avg_selling_price DESC;

```

Finding

Category performance is polarized across three distinct patterns:

- Low-demand categories with weak revenue contribution
- High-demand but low-monetization categories
- Low-volume, high-value premium segments

Insight

Weak engagement and low retention are driven not only by demand gaps but also by poor monetization in high-volume categories, with the absence of discounting limiting conversion and repeat purchase.

ð These imbalances raise the question of whether inventory allocation is aligned with actual customer demand.

Q10. Is inventory allocation aligned with customer demand across categories?

Approach

Compare inventory distribution with actual sales performance to identify overstocked and understocked categories, and assess inventory efficiency.

Metric

- Inventory share (%) by category
- Sales share (%) by category (units-based)
- **Inventory–Sales Gap (%)**
- Average days to sell (inventory turnover proxy)

Key SQL Excerpt

```
WITH category_level AS (  
  
    SELECT category,  
  
    COUNT (DISTINCT inventory_item_id) AS inventory_unit,  
  
    COUNT (DISTINCT CASE WHEN order_status != 'Returned' THEN order_id END) AS  
sold_unit,  
  
    FROM `just-cosmos-479109-p7.thelook_clean.enriched_and_filtered_data`  
  
    GROUP BY 1),  
  
category_share AS (  
  
    SELECT *,  
  
    SAFE_DIVIDE(inventory_unit,SUM(inventory_unit) OVER ()) AS inventory_share,  
  
    SAFE_DIVIDE((sold_unit),SUM(sold_unit) OVER ()) AS sale_share,  
  
    FROM category_level)  
  
SELECT category,  
  
    ROUND(inventory_share - sale_share ,4) AS inventory_sale_gap,  
  
    ROUND(inventory_share,4) AS inventory_share,  
  
    ROUND(sale_share,4) AS sale_share  
  
FROM category_share  
  
ORDER BY inventory_sale_gap DESC;
```

Note

Due to data limitations, inventory efficiency cannot be directly measured; therefore, the analysis focuses on supply–demand alignment as a proxy.

Finding

Inventory allocation is highly aligned with demand across all categories, with near-zero inventory–sales gaps and no meaningful over- or under-allocation.

Insight

The uniform dominance of slow-moving inventory across all categories suggests that the core issue lies in weak product-level demand rather than category-level misallocation.

đ Balanced allocation confirms supply is not the bottleneck; growth is constrained by weak demand conversion and monetization.